



Security Review For Spiral Stake



Collaborative Audit Prepared For:
Lead Security Expert(s):

Spiral Stake
PeterSR
TIMOH

Date Audited:

April 27 - May 2, 2026

Introduction

Spiral Stake is a non-custodial leverage execution layer for digital assets, built on top of Morpho. It takes one well-understood DeFi primitive - looping (recursive borrow-and-redeposit to amplify exposure) and turns it into a one-click, atomic action through its Flash-Leverage mechanism, across an expanding menu of strategies. Users can amplify stablecoin and RWA yields or take leveraged longs on ETH, BTC, and stocks, all without funding rates.

Scope

Repository: `spiral-stake/v2-core`

Audited Commit: `41ce94e9086f5615763fb9d3e70da3ee31c08ce8`

Final Commit: `ab655f9cd9618465b83246859b374870718157f5`

Files:

- `src/core/FlashLeverage/FlashLeverage.sol`
- `src/core/FlashLeverage/MarketPositionManager.sol`
- `src/core/FlashLeverage/SwapManager.sol`
- `src/core/FlashLeverage/UserProxy.sol`
- `src/core/libraries/Error.sol`
- `src/core/libraries/Math.sol`
- `src/core/libraries/TokenHelper.sol`
- `src/core/structs/LeverageParams.sol`
- `src/core/structs/LeveragePosition.sol`
- `src/core/structs/MarketConfig.sol`
- `src/core/structs/SwapData.sol`
- `src/router/FlashLeverageRouter.sol`

Final Commit Hash

`ab655f9cd9618465b83246859b374870718157f5`

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.

- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	1	6

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue M-1: Partial deleverage locks collateral in Morpho and allows to avoid fee [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-04-spiral-stake-apr-27th/issues/13>

Summary

Deleverage is supposed to close position: withdraw all collateral and repay all debt. But user has option to withdraw collateral partially, in this case user doesn't have option to withdraw remaining collateral. Such partial deleverage also allows to avoid paying fee.

Vulnerability Detail

Flow is following:

- 1) Flashloan is taken for position's loan amount
- 2) Full loan is repayed
- 3) Collateral is withdrawn
- 4) Position is closed

However user has ability to specify how much collateral to withdraw:

```
function deleverage(
    uint256 positionId,
    uint256 amountCollateral,
    SwapData memory swapData,
    uint256 minTokenOut
) external nonReentrant whenNotPaused returns (uint256) {
    ...
    require(position.open, FLError.FlashLeverage__PositionAlreadyClosed());
@>    position.open = false;
    ...
@>    // If user passes 0, use full Morpho collateral (default behavior)
@>    // If non-zero, validate it doesn't exceed actual collateral
@>    uint256 collateralToWithdraw = amountCollateral == 0
        ? morphoPosition.collateral
        : amountCollateral;

    require(
        collateralToWithdraw <= morphoPosition.collateral,
        FLError.FlashLeverage__ExceedsCollateral()
    );

    bytes memory data = abi.encode(
        Action.UNLEVERAGE,
```

```

        user,
        positionId,
        morphoPosition.borrowShares,
@>        collateralToWithdraw,
        swapData,
        minTokenOut
    );

    ...
}

```

That specific amount is then withdrawn from Morpho:

```

function _handleDeleverage(
    uint256 amountLoan,
    bytes memory data
) internal override {
    (
        ,
        address user,
        uint256 positionId,
        uint256 borrowShares,
@>        uint256 amountLeveragedCollateral,
        SwapData memory swapData,
        uint256 minTokenOut
    ) = abi.decode(
        data,
        (Action, address, uint256, uint256, uint256, SwapData, uint256)
    );

    ...
    // Close the position's existing loan, with the flashloan, to withdraw
    → position's required collateral
    _repayAndWithdrawCollateralViaProxy(
        position.userProxy,
        market,
        amountLoan,
@>        amountLeveragedCollateral,
        borrowShares
    );

    // Swap withdrawn collateral -> loan token
    uint256 amountSwappedLoanToken = _swapToken(
        market.collateralToken,
        market.loanToken,
@>        amountLeveragedCollateral,
        swapData,
        minTokenOut
    );

    if (amountLoan > 0) {

```

```

        // Repay the flash loan, with swapped loan token
        _forceApprove(market.loanToken, address(i_morpho), amountLoan);
    }

    uint256 totalAmountReturned;
    if (amountSwappedLoanToken > amountLoan) {
        unchecked {
            totalAmountReturned = amountSwappedLoanToken - amountLoan;
        }
    }
    ...
    // Transfer remaining amount to user
    uint256 amountReturned = (totalAmountReturned - amountFee);
    _transferOut(market.loanToken, user, amountReturned);
}

```

Suppose following example:

- 1) User creates 2x leveraged position with loan = 1000, collateral = 2000
- 2) Then calls deleverage() with amount 1500
- 3) It will repay 1000 loan and withdraw 1500 collateral
- 4) Then convert collateral to loan and repay flashloan
- 5) So in the end transfers 500 loan token to user
- 6) Remaining 500 collateral is still in Morpho
- 7) Collateral can't be withdrawn because position is closed: position.open = false.
The only option is to contact admin and withdraw it via manual mode.

Moreover such vector allows to avoid paying fee. Let's consider following scenario:

- 1) Same position as above: loan = 1000, collateral = 2000
- 2) Yield fee is 10%, assume yield is 10%, so collateral grows to 2200
- 3) It means that yield fee should be $200 * 10\% = 20$
- 4) But user can call deleverage() with amount 1000
- 5) It will calculate that user lost any surplus collateral over loan, because totalAmountReturned = 0 and hence won't take fee:

```

function _handleDeleverage(
    uint256 amountLoan,
    bytes memory data
) internal override {
    ...
    uint256 totalAmountReturned;
    if (amountSwappedLoanToken > amountLoan) {
        unchecked {
            totalAmountReturned = amountSwappedLoanToken - amountLoan;

```

@>

```

    }
}

uint256 amountFee; // Only charge yield fee for correlated assets
if (s_isCorrelated[position.marketId]) {
    // All calculation are in loanToken decimals
    uint256 amountDeposited = position.amountDepositedInLoanToken;

    if (totalAmountReturned > amountDeposited) {
        uint256 yieldGeneratedInLoanToken = totalAmountReturned -
            amountDeposited;
        amountFee = yieldGeneratedInLoanToken.mulDown(s_yieldFee);
        _transferOut(market.loanToken, s_treasury, amountFee);
    }
}

...
}

```

- 6) Then user requests admin to enable manual mode and withdraws remaining 1200 collateral manually without fee.

Impact

When user performs deleverage with partial collateral withdraw, remaining collateral can't be withdrawn. The only option left is emergency path via admin to enable manual mode and then withdraw. Additionally user can try to avoid paying fee via this route, though it's risky.

Code Snippet

<https://github.com/sherlock-audit/2026-04-spiral-stake-apr-27th/blob/main/v2-core/src/core/FlashLeverage/FlashLeverage.sol#L258-L262>

Tool Used

Manual Review

Recommendation

Allow calling `withdrawCollateral()` on closed position. Closed position has collateral in 2 scenarios:

- 1) `deleverage()` didn't withdraw full collateral

- 2) Collateral was supplied directly via Morpho. It will be treated as positive yield, irrelevant here.

```
function withdrawCollateral(
    uint256 positionId,
    uint256 amountWithdraw
) external nonReentrant whenNotPaused validateAmount(amountWithdraw) {
    address user = msg.sender;
    LeveragePosition storage position = s_userLeveragePositions[user][
        positionId
    ];
-   require(position.open, FLError.FlashLeverage__PositionAlreadyClosed());
+   require(position.userProxy != address(0));
```

Update `position.amountDepositedInLoanToken` in `deleverage()` so that consequent `withdrawCollateral()` takes fee correctly:

```
function _handleDeleverage(
    uint256 amountLoan,
    bytes memory data
) internal override {
    ...
    uint256 amountFee; // Only charge yield fee for correlated assets
    if (s_isCorrelated[position.marketId]) {
        // All calculation are in loanToken decimals
        uint256 amountDeposited = position.amountDepositedInLoanToken;

        if (totalAmountReturned > amountDeposited) {
            uint256 yieldGeneratedInLoanToken = totalAmountReturned -
                amountDeposited;
            amountFee = yieldGeneratedInLoanToken.mulDown(s_yieldFee);
            _transferOut(market.loanToken, s_treasury, amountFee);
        }
    }

    // Transfer remaining amount to user
    uint256 amountReturned = (totalAmountReturned - amountFee);
    _transferOut(market.loanToken, user, amountReturned);

    position.amountReturnedInLoanToken = amountReturned;
+   position.amountDepositedInLoanToken = position.amountDepositedInLoanToken >
↪ amountReturned ? position.amountDepositedInLoanToken - amountReturned : 0;
    emit LeveragePositionClosed(user, positionId, amountReturned);
}
```

Discussion

Bhimgouda

Valid Medium, as user gets to choose the amount, so stuck collateral is a real path only when user explicitly chooses to withdraw less than full collateral. Even passing 0 withdraws everything correctly.

The fee bypass is self-punishing without admin cooperation, as user loses the stuck collateral to avoid paying the yield fee.

Context on the parameter: we previously fetched collateral directly from Morpho (`morphoPosition.collateral`), but that created a front-running DoS where an attacker could supply 1 wei of collateral to the proxy, causing the pre-built swap calldata to mismatch and revert (as partial deleverage is forbidden as it helps with bypassing fees)

The `amountCollateral` parameter was the fix for that. Open to a better approach that prevents both the DoS and partial deleverage.

TIMOH593

@Bhimgouda updated mitigation

Bhimgouda

@Bhimgouda updated mitigation

The recommended mitigation looks good to me. This was skipped earlier, and I've acknowledged the issue for now, but I'm happy to proceed with this fix—as long as updating `amountDepositedInLoanToken` in `deleverage` and allowing `withdrawCollateral` for a closed position do not introduce a new vulnerability or enable a fee bypass

Bhimgouda

fix commit - <https://github.com/spiral-stake/v2-core/commit/a3e93ec263e2dcf0c765cbbadc5ba8adfd917970>

Issue L-1: Yield Fee Path Arbitrage via Oracle-DEX Price Divergence [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-04-spiral-stake-apr-27th/issues/16>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`withdrawCollateral()` computes yield using the oracle price, while `deleverage()` computes yield from the actual DEX swap output. When the oracle lags below the live market price, a user can route their exit through `repay() + withdrawCollateral()` and pay a systematically lower yield fee than `deleverage()` would charge on an identical position.

Vulnerability Detail

The protocol has two exit paths for correlated leveraged positions, and they use different price sources to determine how much yield the user generated:

`withdrawCollateral()` — **oracle-priced yield** (`FlashLeverage.sol:503-530`):

```
uint256 amountCollateralInLoanToken = getCollateralValueInLoanToken(
    market,
    morphoPosition.collateral // valued at current oracle price
);
uint256 netPositionValue = amountCollateralInLoanToken - amountLoan;
yieldGeneratedInLoanToken = netPositionValue - position.amountDepositedInLoanToken;
```

`deleverage()` — **DEX-priced yield** (`FlashLeverage.sol:888-891`):

```
uint256 yieldGeneratedInLoanToken = totalAmountReturned - amountDeposited;
// totalAmountReturned = actual DEX swap output - flash loan repaid
```

Chainlink oracles update on a heartbeat or deviation threshold, not continuously, so they routinely lag the live DEX market price. When $\text{oracle} < \text{DEX price}$, `withdrawCollateral()` underestimates the collateral value, computes less yield, and charges less fee.

The amplification is the key mechanic: the oracle-DEX gap applies to the **full collateral value**, but the fee is charged only on the **profit slice**. For a position 10% in profit with a 1% oracle lag, the fee discrepancy is approximately 10% – because 1% of the full collateral value is roughly equal to the entire profit portion.

Concrete example (2× leveraged position):

- Open: deposit 100 WETH, protocol flash-loans 100 WETH, buys 200 wstETH at 1.0. `amountDepositedInLoanToken = 100`, `debt = 100 WETH`.
- Price appreciates 10%. Oracle = 1.1 WETH/wstETH. DEX = 1.111 (1% above oracle).

repay() + *withdrawCollateral()* path:

1. User calls `repay(100 WETH)` – debt cleared, `amountDepositedInLoanToken` becomes 200.
2. `withdrawCollateral(200 wstETH)`: oracle value = 220 WETH, yield = $220 - 200 = 20$ WETH, fee = **2 WETH**.
3. User sells 200 wstETH on DEX at 1.111 = 222.2 WETH. Net after repay cost: **-120.2 WETH**.

deleverage() path:

1. DEX swap output = $200 \times 1.111 = 222.2$ WETH. `totalAmountReturned` = 122.2 WETH.
2. Yield = $122.2 - 100 = 22.2$ WETH, fee = **2.22 WETH**.
3. Net: **-119.98 WETH**.

The user saves ~0.22 WETH (10% of the fee) by choosing the oracle-lagged path. The optimal path is trivially determinable by comparing the oracle price to any DEX quote before submitting the transaction. Note that the `withdrawCollateral()` path requires the user to front the debt capital upfront via `repay()`, rather than relying on a flash loan – this is a real constraint but available to any user with sufficient loan token liquidity.

Impact

Users can systematically underpay yield fees by selecting the lower-fee exit path whenever the oracle lags the DEX price. The fee evasion magnitude scales with the ratio of total collateral value to profit – lower-margin positions see greater amplification. Since the optimal path is determinable from public on-chain data before committing to any transaction, sophisticated users will always route through `withdrawCollateral()` during oracle lag, permanently reducing protocol fee revenue on correlated markets.

Tool Used

Manual Review

Recommendation

Standardize yield computation on a single price source across both exit paths. The most consistent fix is to apply oracle-based yield calculation inside `_handleDeleverage()` as well, so the fee is independent of DEX swap output:

```
// In _handleDeleverage(), replace DEX-output yield with oracle-based yield:
uint256 oracleCollateralValue = getCollateralValueInLoanToken(market,
    ↪ initialCollateral);
uint256 yieldGeneratedInLoanToken = oracleCollateralValue > amountDeposited
    ? oracleCollateralValue - amountDeposited
```

```
        : 0;  
amountFee = yieldGeneratedInLoanToken.mulDown(s_yieldFee);
```

Alternatively, record the yield fee basis at position-open time and use it for both paths, removing any real-time price dependency at exit.

Discussion

Bhimgouda

Valid observation, but this is intended behavior and Low severity at most.

WithdrawCollateral uses oracle price because there's no swap involved. Deleverage uses actual swap output because we want to tax on final realized profit after slippage ... charging fees on swap proceeds the user never received would be unfair.

The repay + withdrawCollateral path actually benefits users beyond just fee savings ... there's no swap slippage at all. If the user manually unstakes and gets 1:1, they keep more yield. But this requires upfront capital to repay debt, time required to unstake, multiple transactions, and manual token management versus a single deleverage call. The tradeoff is real and the choice is the user's.

Standardizing on oracle price for deleverage (as recommended) would mean charging fees on unrealized value that gets lost to slippage, users would pay fees on profit they didn't actually receive.

Acknowledged as a known design tradeoff between the two exit paths.

Issue L-2: Share-based repayment is currently blocked [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-04-spiral-stake-apr-27th/issues/17>

Summary

`_morphoRepay` contains a logic/documentation mismatch: the function is documented to support both repay-by-amount and repay-by-shares modes per Morpho Blue's interface, yet the entire repayment path is guarded by `if (amount > 0)`. The pure shares-repayment branch (`amount == 0 && borrowShares > 0`) is a silent no-op.

Vulnerability Detail

In the abstract contract `MarketPositionManager`, the function `_morphoRepay` is explicitly documented to handle both repayment modes required by Morpho:

- Repay-by-amount: `borrowShares == 0, amount > 0`
- Repay-by-shares: `amount == 0, borrowShares > 0` (with the NatSpec note: "When repaying by shares, amount must be 0 per Morpho's interface.")

However, the implementation gates **all** logic (approval + `i_morpho.repay` call) behind:

```
if (amount > 0) { ... }
```

When `amount == 0 && borrowShares > 0`, the function returns `(0, 0)` without executing anything. This breaks the documented API contract.

The concrete inheriting contract `FlashLeverage` never exercises the pure-shares path (it always passes `amount > 0`), making the bug latent today. Nevertheless, the abstract's public interface and NatSpec promise both modes work, creating a maintainability and future-proofing issue.

Impact

- No immediate security or fund-loss impact on the deployed `FlashLeverage` system (all current flows are unaffected).
- Violates the principle of least surprise and the documented contract.
- Future extensions, forks, or new inheriting contracts that rely on the shares-only mode (e.g., exact-position closure without pre-computing amount, or partial repay-by-shares) will silently fail, leading to incorrect position state, failed deleverages, or stuck flash-loan callbacks.

Code Snippet

```
function _morphoRepay(
  address userProxy,
  MarketParams memory marketParams,
  uint256 amount,
  uint256 borrowShares
) internal returns (uint256 assetsRepaid, uint256 sharesRepaid) {
  if (amount > 0) { // ← BUG: pure shares mode (amount==0 && borrowShares>0) is
    ↪ unreachable
    _forceApprove(marketParams.loanToken, address(i_morpho), amount);

    address onBehalf = userProxy;
    (assetsRepaid, sharesRepaid) = i_morpho.repay(
      marketParams,
      borrowShares == 0 ? amount : 0,
      borrowShares,
      onBehalf,
      hex""
    );
  }
}
```

(See also: NatSpec comment above the function.)

Tool Used

Manual Review

Recommendation

Update the natspec for the correct behaviour.

Discussion

Bhimgouda

Every call to `_morphoRepay` in `FlashLeverage` passes `amount > 0`. `repay()` has `validateAmount(amountRepay)` enforcing non-zero, and `_handleDeleverage` always has a non-zero flash loan amount when borrow shares exist. The pure shares-only path is never reached and not needed for our use case.

I have updated the NatSpec to clarify the expected usage. The recommended code change adds unnecessary complexity for a path we don't use.

<https://github.com/spiral-stake/v2-core/tree/fixes>

Issue L-3: Retroactive `s_isCorrelated` Flag Flip Changes Fee Model for Existing Positions [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-04-spiral-stake-apr-27th/issues/18>

Summary

`addSupportedMarkets()` can overwrite `s_isCorrelated[marketId]` at any time for any market, including markets that already have open positions. Because `s_isCorrelated` is read globally at every fee-charging operation rather than stored per-position, all open positions on the affected market immediately switch fee models – with no notice and no recourse for the user.

Vulnerability Detail

`s_isCorrelated` is a global mapping that the protocol reads at four distinct points in the position lifecycle:

- `repay()` (L402): determines whether basis is decremented or capped at 0
- `withdrawCollateral()` (L501): gates whether a yield fee is charged on withdrawal
- `_handleDeleverage()` (L884): gates whether a yield fee is collected at close
- `_handleLeverage()` (L915): determines whether a deposit fee is charged at open

`addSupportedMarkets()` overwrites this value unconditionally:

There is no check for whether the market has open positions, no migration path, and no per-position snapshot of the flag at open time.

Flip from correlated → non-correlated: A user opens a `wstETH/WETH` position expecting a yield fee (10%) at close and no deposit fee at entry (they paid none when the position was opened). After the flip, any subsequent `repay()` or `supplyCollateral()` call charges a deposit fee on the supplied amount (L402 path). At close, `_handleDeleverage()` skips yield fee collection entirely.

Flip from non-correlated → correlated: A user opens a `WBTC/USDC` position having paid the deposit fee at entry. After the flip, their position now attracts a yield fee (up to 10%) at close – a fee they did not agree to and did not price into their position sizing. The deposit fee already paid does not offset this new fee.

Impact

Users may pay more fees than the fee model in effect when they opened their position, with no on-chain mechanism to close before the change takes effect. The economic impact is bounded by the yield fee cap (10% of profit) and is conditional on the owner

changing the flag, but the change is unilateral and can be applied to any market regardless of open position count.

Tool Used

Manual Review

Recommendation

Store `isCorrelated` per-position at open time in the `LeveragePosition` struct, so each position is governed by the fee model in effect when it was created:

```
struct LeveragePosition {  
    // ... existing fields ...  
    bool isCorrelated; // snapshot at open time  
}
```

Read `position.isCorrelated` in `repay()`, `withdrawCollateral()`, and `_handleDeleverage()` instead of `s_isCorrelated[position.marketId]`. The global mapping continues to govern new positions opened after any flag change.

Discussion

Bhimgouda

Valid concern. Rather than per-position snapshots, preventing the issue at the source, `addSupportedMarkets` will now revert if the market already exists. This prevents any re-configuration of `isCorrelated` on markets with open positions. Its a simpler fix, same guarantee.

fix commit - <https://github.com/spiral-stake/v2-core/commit/dfc5ecf05e98ee778d8d918e556bfeedaa213ald>

TIMOH593

LGTM

Issue L-4: FlashLeverage.repay() takes fee on excess amount [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-04-spiral-stake-apr-27th/issues/19>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

FlashLeverage.repay() firstly takes fee on whole amount user submits, and then returns excess funds if there are any.

Vulnerability Detail

User can repay debt on his position, there are 2 ways: 1) submit debt amount to repay in assets; 2) or submit amount to repay in shares.

It's possible to send excess amount if user repays using shares. Function _morphoRepay() doesn't use assets amount if there is borrowShares specified:

```
function repay(
    address user,
    uint256 positionId,
    uint256 amountRepay,
    uint256 borrowShares // Can be 0, mostly used for full loan repayment
) external nonReentrant whenNotPaused validateAmount(amountRepay) {
    ...

    amountRepay = _chargeDepositFeeIfNonCorrelated(
        market.loanToken,
        position.marketId,
        amountRepay
    );

    (uint256 amountRepaid, ) = _morphoRepay(
        position.userProxy,
        market,
        amountRepay,
        borrowShares
    );
    position.amountDepositedInLoanToken += amountRepaid;

    uint256 excess = amountRepay - amountRepaid;
    if (excess > 0) {
        _transferOut(market.loanToken, msg.sender, excess);
    }
    ...
}
```

```

    }

    function _morphoRepay(
        address userProxy,
        MarketParams memory marketParams,
        uint256 amount,
        uint256 borrowShares
    ) internal returns (uint256 assetsRepaid, uint256 sharesRepaid) {
        if (amount > 0) {
            _forceApprove(marketParams.loanToken, address(i_morpho), amount);

            address onBehalf = userProxy;
            (assetsRepaid, sharesRepaid) = i_morpho.repay(
                marketParams,
                borrowShares == 0 ? amount : 0,
                borrowShares,
                onBehalf,
                hex""
            );
        }
    }
}

```

So following situation is possible:

- 1) User's debt is 1000 assets, or 1000 shares for simplicity
- 2) User repays 1000 shares and submits 1200 assets
- 3) Fee is taken on whole 1200 assets submitted instead of 1000
- 4) Debt is repaid and excess amount is returned to user.

Impact

If user repays debt submitting `borrowShares`, fee is taken on excess amount submitted.

Code Snippet

<https://github.com/sherlock-audit/2026-04-spiral-stake-apr-27th/blob/main/v2-core/src/core/FlashLeverage/FlashLeverage.sol#L431-L448>

Tool Used

Manual Review

Recommendation

Ideally it should take fee only on actual amountRepaid after repayment. At least document this behaviour.

Discussion

Bhimgouda

Valid observation but negligible impact. The fee overcharge only applies to the excess buffer on repay-by-shares, which is typically a few wei of rounding. The overcharge is 1% of that excess - dust level. Not worth the added complexity of restructuring the fee flow. Acknowledged & the behaviour will be documented

Issue L-5: Natspec mismatch in FlashLeverage.validateUser [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-04-spiral-stake-apr-27th/issues/20>

Summary

User can be address(0) contrary to NatSpec

Vulnerability Detail

NatSpec states that it reverts on zero address, however it won't if msg.sender is approved operator:

```
/**
 * @notice Validates the user address and the caller is authorized
 * @param user The address of the user for whom the action is being performed
@> * @dev Reverts if user is zero address or caller is neither the user nor an
↪ approved operator
 */
modifier validateUser(address user) {
    require(
        msg.sender == user || s_approvedOperators[msg.sender],
        FLError.FlashLeverage__NotApprovedOperator()
    );
    _;
}
```

So actually it's possible to create position for 0 address.

Code Snippet

<https://github.com/sherlock-audit/2026-04-spiral-stake-apr-27th/blob/main/v2-core/src/core/FlashLeverage/FlashLeverage.sol#L113-L124>

Tool Used

Manual Review

Recommendation

Consider updating either NatSpec or code.

Discussion

Bhimgouda

Valid ... already identified in a prior audit (Solace L-3). added `require(user != address(0))` to the `validateUser` modifier.

fix commit - <https://github.com/spiral-stake/v2-core/commit/a90da579b2cf6c638ec78a992b69f85943f4b521>

TIMOH593

LGTM

Issue L-6: Full repay can be grieved [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-04-spiral-stake-apr-27th/issues/21>

Summary

Griever can frontrun full repay transaction by repaying 1 share. It makes user's transaction revert because it tries to repay more debt than it has.

Vulnerability Detail

Let's take a look on how repay works. Interest accrues each block, so user can specify full debt in borrowShares. Still he needs to specify sufficient amount of assets amountRepay for approval:

```
function repay(
    address user,
    uint256 positionId,
    uint256 amountRepay,
    @> uint256 borrowShares // Can be 0, mostly used for full loan repayment
) external nonReentrant whenNotPaused validateAmount(amountRepay) {
    LeveragePosition storage position = s_userLeveragePositions[user][
        positionId
    ];

    require(position.open, FLError.FlashLeverage__PositionAlreadyClosed());

    MarketParams memory market = s_markets[position.marketId];

    _transferIn(market.loanToken, msg.sender, amountRepay);
    ...
    @> (uint256 amountRepaid, ) = _morphoRepay(
        position.userProxy,
        market,
        amountRepay,
        borrowShares
    );
    ...
}

function _morphoRepay(
    address userProxy,
    MarketParams memory marketParams,
    uint256 amount,
    uint256 borrowShares
) internal returns (uint256 assetsRepaid, uint256 sharesRepaid) {
    if (amount > 0) {
```

```

@>         _forceApprove(marketParams.loanToken, address(i_morpho), amount);

        address onBehalf = userProxy;
        (assetsRepaid, sharesRepaid) = i_morpho.repay(
            marketParams,
@>         borrowShares == 0 ? amount : 0,
@>         borrowShares,
            onBehalf,
            hex""
        );
    }
}

```

Morpho reverts if user tries to repay more debt than position has:

```

function repay(
    MarketParams memory marketParams,
    uint256 assets,
    uint256 shares,
    address onBehalf,
    bytes calldata data
) external returns (uint256, uint256) {
    Id id = marketParams.id();
    require(market[id].lastUpdate != 0, ErrorsLib.MARKET_NOT_CREATED);
    require(UtilsLib.exactlyOneZero(assets, shares),
        ↪ ErrorsLib.INCONSISTENT_INPUT);
    require(onBehalf != address(0), ErrorsLib.ZERO_ADDRESS);

    _accrueInterest(marketParams, id);

    if (assets > 0) shares = assets.toSharesDown(market[id].totalBorrowAssets,
        ↪ market[id].totalBorrowShares);
    else assets = shares.toAssetsUp(market[id].totalBorrowAssets,
        ↪ market[id].totalBorrowShares);

@>     position[id][onBehalf].borrowShares -= shares.toUint128();
    market[id].totalBorrowShares -= shares.toUint128();
    ...
}

```

Consider following example:

- 1) User's debt is 1500 assets or 1000 shares. User wants to repay full debt
- 2) User specifies borrowShares = 1000 and calls repay()
- 3) Griefer frontruns and repays 1 share directly on Morpho
- 4) User's repay() transaction reverts

Same if user specifies assets instead of shares. User has no meanings to close debt in full

at once.

Impact

Griefer can make full repay transaction revert by frontrunning.

Code Snippet

<https://github.com/sherlock-audit/2026-04-spiral-stake-apr-27th/blob/main/v2-core/src/core/FlashLeverage/FlashLeverage.sol#L415-L419>

<https://github.com/sherlock-audit/2026-04-spiral-stake-apr-27th/blob/main/v2-core/src/core/FlashLeverage/MarketPositionManager.sol#L187-L205>

Tool Used

Manual Review

Recommendation

Consider using specific flag for full repay. If that's full repay, fetch `borrowShares` directly from Morpho.

Discussion

Bhimgouda

Valid – fixing this. Full repay now supports `borrowShares = type(uint256).max` which fetches actual shares from Morpho at execution time. Front-runner repaying 1 share no longer causes a revert since we read the current (reduced) shares before calling Morpho. Excess tokens are refunded to caller.

fix commit - <https://github.com/spiral-stake/v2-core/commit/9ccd288f8d4743c098c3cca76171a2c81741f87d>

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.